



END-TO-END EXECUTION WALKTHROUGH

One Query, Every Concept From This Series

Driver

DAG

Catalyst

Shuffle

Tasks

The Complete Picture — 7 Slides

Swipe to start learning →

0 THE QUERY

```
sales = spark.read.parquet("s3://.../sales")
customers = spark.read.parquet("s3://.../customers")

result = sales.filter(col("amount") > 100) \
    .join(broadcast(customers), "cust_id") \
    .groupBy("region").agg(sum("amount")) \
    .orderBy("region")

result.write.parquet("s3://.../output")
```

What We'll Trace

Every line above triggers something specific inside Spark. Over the next five slides, we follow it from the Driver all the way to disk.

Read

Filter

Broadcast Join

GroupBy

OrderBy

Write

1 NOTHING RUNS YET

Every transformation so far — filter, join, groupBy, orderBy — only extends a plan sitting in Driver memory.

What's Happening on the Driver

1 Each chained call extends the **Unresolved Logical Plan**



2 Catalyst's **Analysis** phase resolves columns/types against the catalog



3 **Logical Optimization** pushes amount > 100 down into the Parquet scan and prunes unused columns



4 Still **zero data** has moved — all of this runs on the Driver alone

Interview Tip: This is Lazy Evaluation (Part 3) and the Catalyst Optimizer (Part 5) working before a single Task exists.

2 CHOOSING HOW TO RUN IT

The `.write.parquet()` call is the Action — the exact moment a Job gets submitted.

From Plan to Submitted Job

1 The `broadcast()` hint makes Catalyst pick **Broadcast Hash Join** over Sort-Merge — no shuffle for the join



2 `.write.parquet()` runs — the Job is submitted



3 DAGScheduler walks the plan backwards, building the DAG



4 **groupBy** forces a shuffle → the DAG splits into **2 Stages** right there

Interview Tip: Notice the join added zero Stage boundaries — only the groupBy shuffle did.

3 PARALLEL EXECUTION BEGINS

With Stages defined, Spark fans work out across every Executor the Cluster Manager already allocated.

Turning Stages Into Work

1 Stage 1 (scan + filter + broadcast join) — one Task per input Parquet partition



2 The small **customers** DataFrame is broadcast once to every Executor before Stage 1 starts



3 Stage 2 (post-shuffle aggregation + sort) — one Task per **shuffle partition**



4 All of this runs on Executors the **Cluster Manager** allocated before the Job even started

Interview Tip: Parts 1, 4, 6 and 8 of the series all meet in this one Job — Cluster Manager, Tasks, Partitioning, and Broadcast.

4 FROM COMPUTE TO THE FINISH LINE

The last leg is where memory pressure, code generation, and disk output all come together.

Landing the Result

1 Per-region sums live in **Execution Memory** on each Executor



2 A skewed region's Task may **spill to disk** under memory pressure



3 **Whole-Stage CodeGen** fuses filter + join + aggregate into one compiled loop per partition



4 Once every Task in Stage 2 finishes, results write to Parquet — the Job is complete

Interview Tip: A slow run here means checking Parts 6 and 7 first — shuffle skew or memory spill, in that order.

Every part of this series, mapped to the walkthrough above

SERIES PART	WHERE IT SHOWED UP HERE
1. Architecture	Driver planned it, Executors ran it
2. Execution Flow	The exact chain we just traced
3. Lazy Evaluation	Nothing ran until .write()
4. DAG/Jobs/Stages/Tasks	1 Job → 2 Stages → many Tasks
5. Catalyst Optimizer	Pushdown, pruning, join choice
6. Partitioning & Shuffle	The groupBy shuffle boundary
7. Memory Management	Aggregation state + spill risk
8. Broadcast Variables	customers sent once per Executor
9. Performance Tuning	Broadcast hint kept it shuffle-light

You've completed the Spark Internals series!

@chintapradeep

Spark · Python · SQL · Interview Prep